
# *Crossplane vs Kubebuilder*

paradigms, abstraction, complexity, extensibility and performance comparison with a common use case implementation

Infrastructures for Cloud Computing and Big Data M
Lorenzo De Luca
0001240461

# Kubernetes Operators

A Kubernetes Operator extends the Kubernetes API with a Custom Resource(CR) and a controller. Operators implement the reconciliation loop pattern: they observe Custom Resources through the Kubernetes API server, compare desired and actual state, and perform actions until convergence.

Building operators can be approached at different levels of abstraction:

**Low-level**: the developer writes all the controller logic

**High-level**: the developer describes the desired infrastructure API and composition logic, while existing controllers/providers perform reconciliation.


Both approaches solve the same problem of automating resource lifecycle but target different users and different tradeoffs.

# Research Objective

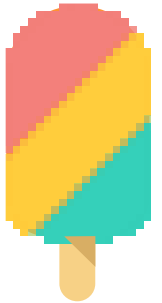
Compare Crossplane and Kubebuilder to compare the two paradigms for building Kubernetes Operators.

Will be evaluated:

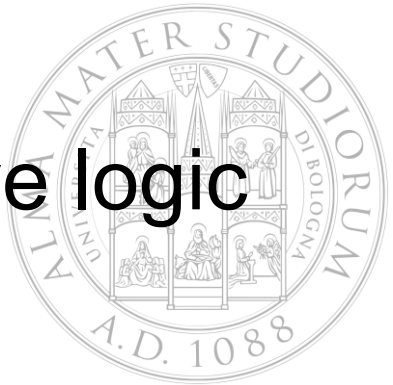
- Architectural model and abstraction level
- Development complexity and extensibility
- Internal optimizations adopted by each framework
- Resource consumption

*does the convenience of the extra abstraction layer of Crossplane cost more in resources/latency than it saves in development effort, compared to a hand-written Kubebuilder controller?*

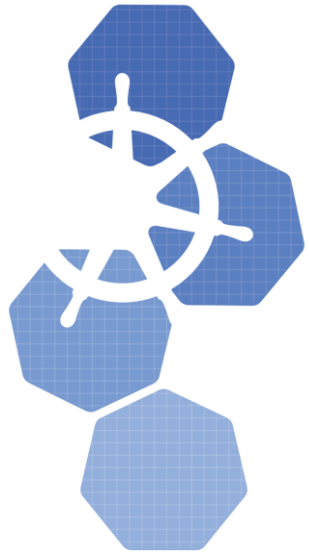
# Crossplane



- Universal Control Plane
- Declarative composition model
- Strong for composition and platform engineering
- Logic is mostly declarative
- Strength: consistency and reuse
- Limitation: less flexible for complex imperative logic



# Kubebuilder



- Low-level Go SDK
- Imperative control model
- Strong for writing specialized controllers from scratch.
- Logic in Go
- Strength: maximum flexibility
- Limitation: higher coding and maintenance effort

# Architecture

## Crossplane

User applies a **Claim** or **Composite Resource**

⬇

Crossplane composition controller observes the resource

⬇

Composition is rendered into composed **Managed Resources**

⬇

**Providers reconcile** each Managed Resource against Kubernetes or external systems

⬇

**Status aggregated bottom-up**:
Provider -> MR -> XR → Claim

## Kubebuilder

User applies a **Custom Resource**

⬇

Controller-runtime informer watches the API server

⬇

Events are added to a work queue

⬇

**Reconcile()** executes custom Go logic and **directly** creates/updates Kubernetes or external resources

⬇

**Status aggregated bottom-up**:
Provider -> MR -> XR → Claim

# Features comparison

| Aspect | Crossplane | Kubebuilder |
|---|---|---|
| Abstraction | High(declarative) | Low(imperative) |
| Architecture | Multi-layer | Single controller |
| Development effort | Lower | Higher |
| Extensibility | Providers, Functions, patching | Arbitrary code, any library |
| Flexibility | Limited | Very high |
| Reusability | High | Low |
| Performance | Higher overhead | More efficient |
| Debugging | Harder | Easier(centralized) |
| Best | Kubebuilder | Less layers, more control |

# Optimizations

Crossplane:

- Provider-level connection reuse/pooling toward external APIs
- Leader election prevents duplicate reconciliation in multi-replica setups
- Multiple controllers run concurrently (core + each provider + functions)

Kubebuilder

- Informer cache: reconciler reads from local cache, not the API server
- Work queue with exponential backoff on repeated failures
- Managed Resource state cached; reconciliation skipped when observed state already matches desired state

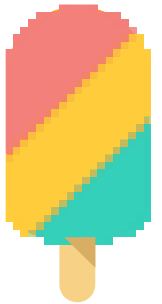
# So… Which one is better? It depends…

| Scenario | Better choice | Reason |
|---|---|---|
| Standard infrastructure composition | Crossplane | Declarative and reusable |
| Internal Developer Platform | Crossplane | User-friendly claims and abstractions |
| Multi-resource self-service APIs | Crossplane | Composition model fits well |
| Complex conditional logic | Kubebuilder | Full control in Go |
| Custom external API integration | Kubebuilder | Any Go library can be used |
| Fine-grained lifecycle control | Kubebuilder | Reconciliation is fully programmable |
| Provisioning latency | Kubebuilder | Fewer controller layers |
| Debuggability | Kubebuilder | Less layers, more control |

# *Kafka Topic Self-Service*

https://github.com/lorenzodeluca/kafka-operator-clash

# Common goal



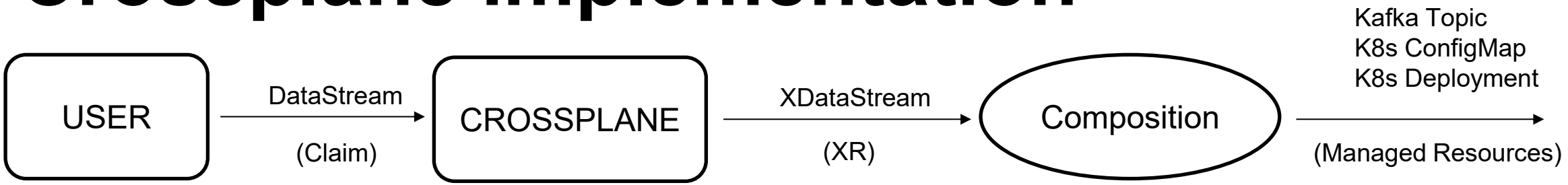
The goal for the implemented operators is to create a new Kafka topic. Instead of manually creating all required resources, the user declares a single object called *DataStream*.

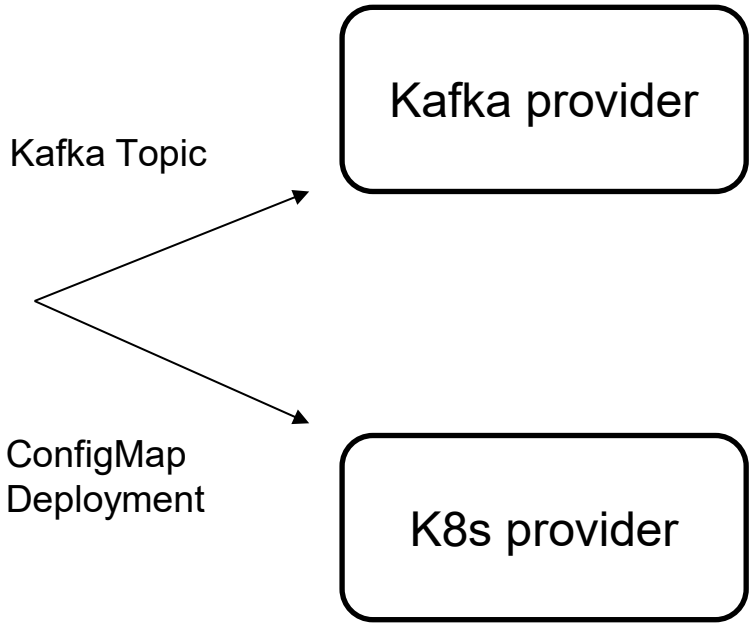
When a *DataStream* is created, the operator must automatically provision and reconcile a ready-to-use data pipeline: a Kafka topic where data can flow, a configuration file with the connection details, and a small consumer application that reads the connection details generated by the operator.

DataStream example:

```yaml
apiVersion: messaging.lorenzodeluca.it/v1alpha1
kind: DataStream
metadata:
  name: lorenzo-sandbox-stream
  namespace: default
spec:
  topicName: "user-telemetry-data"
  partitions: 2
  replicationFactor: 1
```

# Crossplane implementation



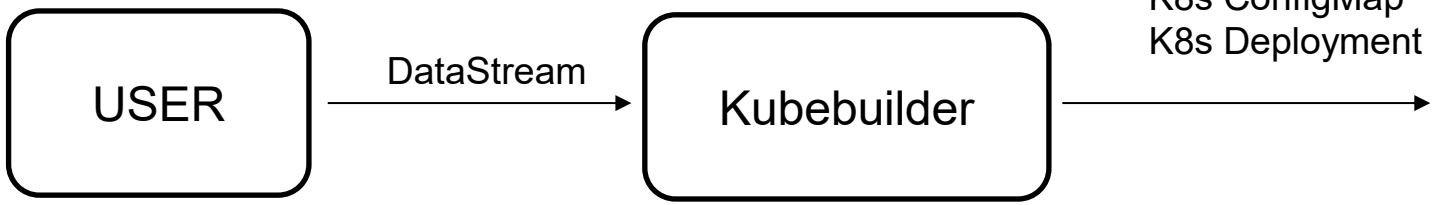

api/datastream_xrd.yaml:

```yaml
apiVersion: apiextensions.crossplane.io/v1
kind: CompositeResourceDefinition
metadata:
  name: xdatastreams.messaging.lorenzodeluca.it
spec:
  group: messaging.lorenzodeluca.it
  names:
    kind: XDataStream
    plural: xdatastreams
  claimNames:
    kind: DataStream
    plural: datastreams
  versions:
    - name: v1alpha1
      served: true
      referenceable: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                topicName:
                  type: string
                partitions:
                  type: integer
                  default: 1
                replicationFactor:
                  type: integer
                  default: 1
              required:
                - topicName
```

compositions/datastream_composition.yaml

```yaml
- step: patch-and-transform
  functionRef:
    name: function-patch-and-transform
  input:
    apiVersion: pt.fn.crossplane.io/v1beta1
    kind: Resources
    resources:
      - name: kafka-topic
        base:
          apiVersion: topic.kafka.crossplane.io/v1alpha1
          kind: Topic
          spec:
            providerConfigRef:
              name: kafka-default
            forProvider:
              partitions: 1
              replicationFactor: 1
        patches:
          - type: FromCompositeFieldPath
            fromFieldPath: spec.topicName
            toFieldPath: metadata.name
          - type: FromCompositeFieldPath
            fromFieldPath: spec.partitions
            toFieldPath: spec.forProvider.partitions
          - type: FromCompositeFieldPath
            fromFieldPath: spec.replicationFactor
            toFieldPath: spec.forProvider.replicationFactor
```

# Kubebuilder implementation




datastream_types.go:

```go
package v1alpha1

import (
    metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
    "k8s.io/apimachinery/pkg/runtime"
)

// DataStreamSpec defines the desired state of DataStream
type DataStreamSpec struct {
    // TopicName is the name of the Kafka topic to manage.
    // +kubebuilder:validation:MinLength=1
    TopicName string `json:"topicName"`

    // Partitions defines the total partition count for the topic.
    // +kubebuilder:default:=1
    // +kubebuilder:validation:Minimum=1
    Partitions int32 `json:"partitions,omitempty"`

    // ReplicationFactor defines the replication factor.
    // +kubebuilder:default:=1
    // +kubebuilder:validation:Minimum=1
    ReplicationFactor int16 `json:"replicationFactor,omitempty"`
}

```

datastream_controller.go:

```go
func createKafkaTopic(broker, topic string, partitions int32, reps int16) error {
    conn, err := kafka.Dial("tcp", broker)
    if err != nil {
        return fmt.Errorf("dial broker failed: %w", err)
    }
    defer conn.Close()

    controller, err := conn.Controller()
    if err != nil {
        return fmt.Errorf("cannot get kafka controller: %w", err)
    }

    controllerAddr := net.JoinHostPort(controller.Host, strconv.Itoa(controller.Port))
    controllerConn, err := kafka.Dial("tcp", controllerAddr)
    if err != nil {
        controllerConn, err = kafka.Dial("tcp", broker)
        if err != nil {
            return fmt.Errorf("cannot dial kafka controller (%s) nor broker fallback (%s): %w", controllerAddr, broker, err)
        }
    }
    defer controllerConn.Close()

    err = controllerConn.CreateTopics(kafka.TopicConfig{
        Topic:             topic,
        NumPartitions:     int(partitions),
        ReplicationFactor: int(reps),
    })
    if err != nil {
        l := strings.ToLower(err.Error())
        if strings.Contains(l, "already exists") || strings.Contains(l, "topic already exists") {
            return nil
        }
        return err
    }
    return nil
}
```

# Performance results

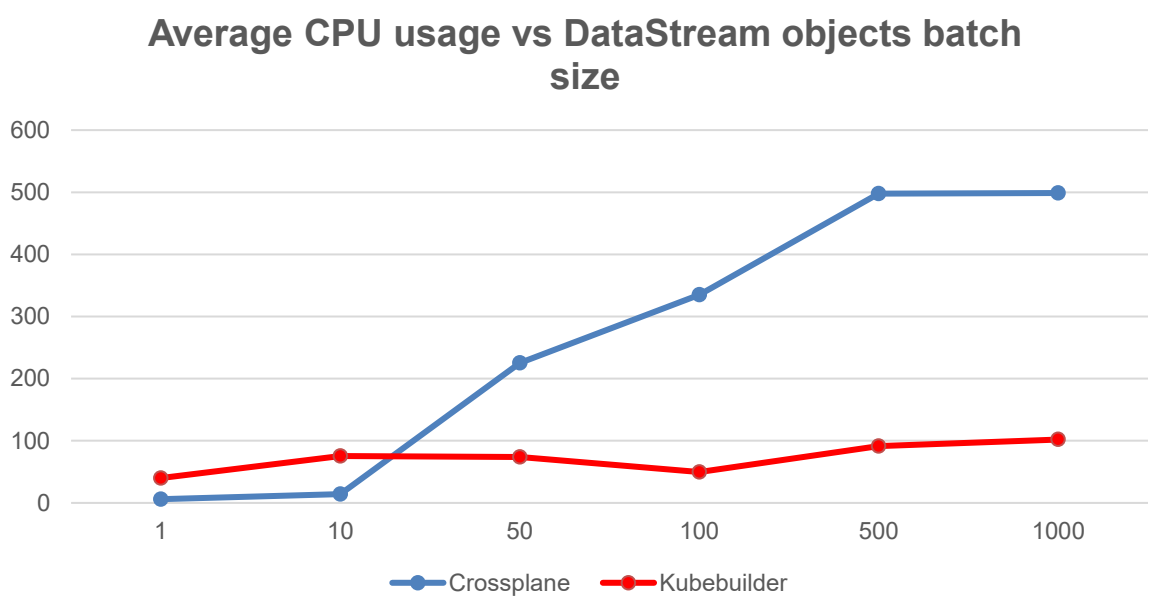
| Metric | Crossplane | Kubebuilder |
|---|---|---|
| Lines of code written* | 165 | 270 |
| Manually edited files | 7 | 2 |
| Avg time to ready state (seconds)** | 6 | 2 |

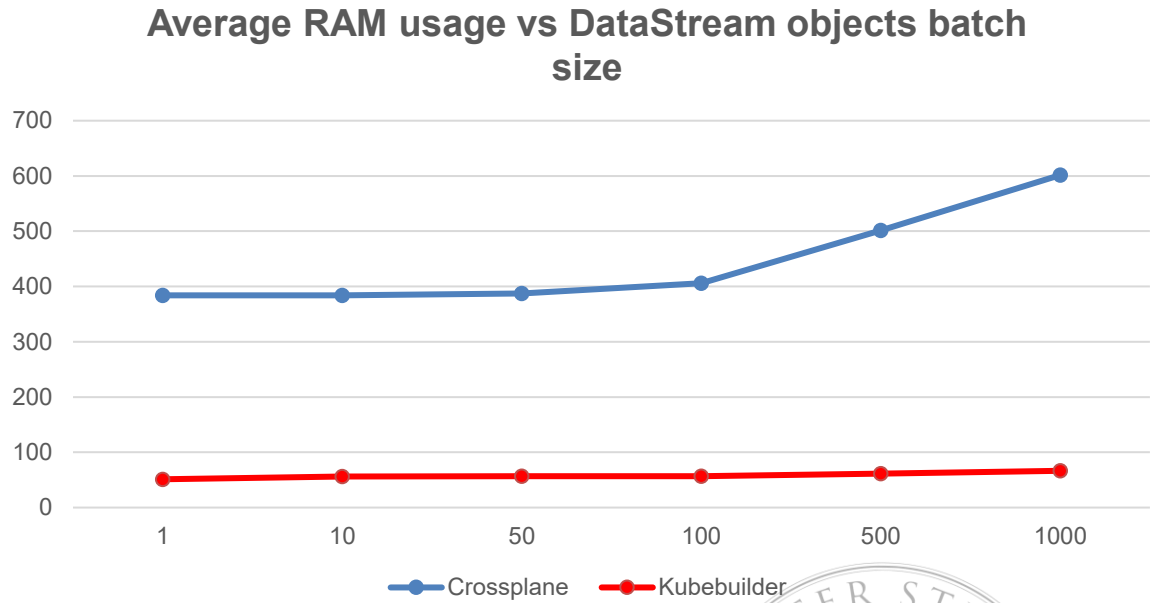
- Excluding generated code and provider installation/configuration manifests. The metric is indicative only since Crossplane shifts complexity to providers and platform configuration, while Kubebuilder concentrates complexity in the controller code.

** The measured time is the elapsed time between applying the DataStream resource and observing the expected ready state.

Environment:
- Debian vps with 8 cores cpu, 8Gb ram, 75Gb ssd
- Local k3s cluster
- Single Kafka broker
- Controllers/providers already running
- Crossplane components running in-cluster
- Kubebuilder controller running as host process with ’make run’
- Metrics collected through `kubectl top` for pods and ’/proc’ for Kubebuilder

# Resource consumption

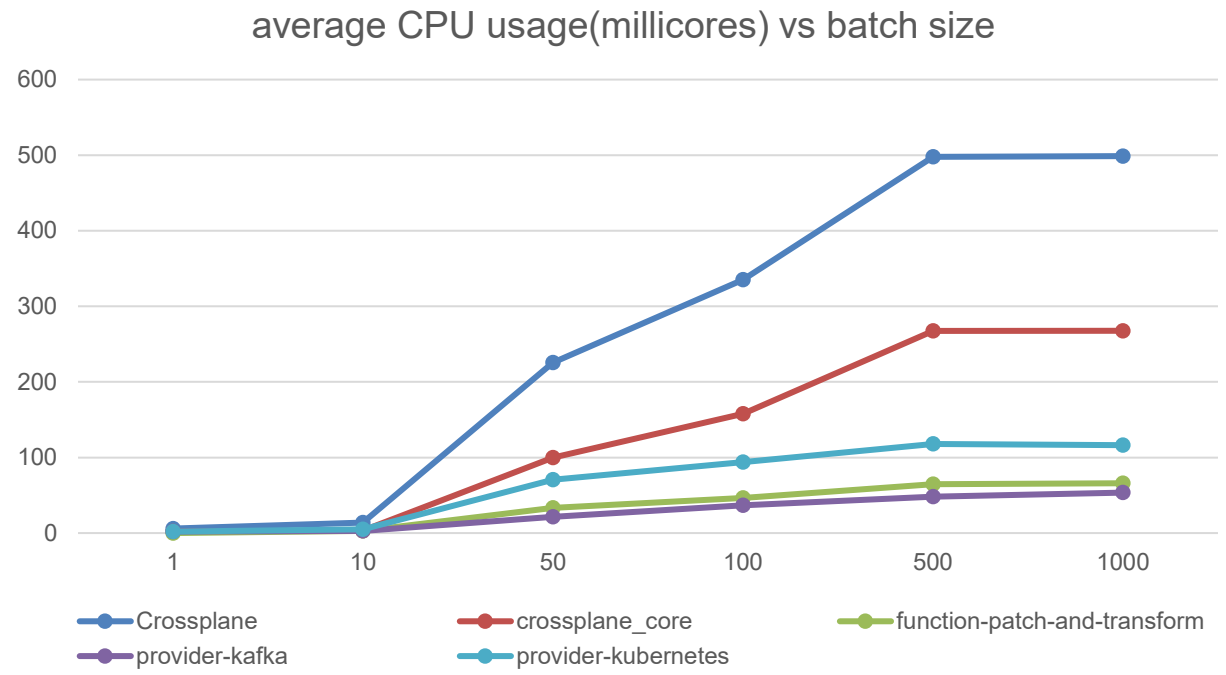


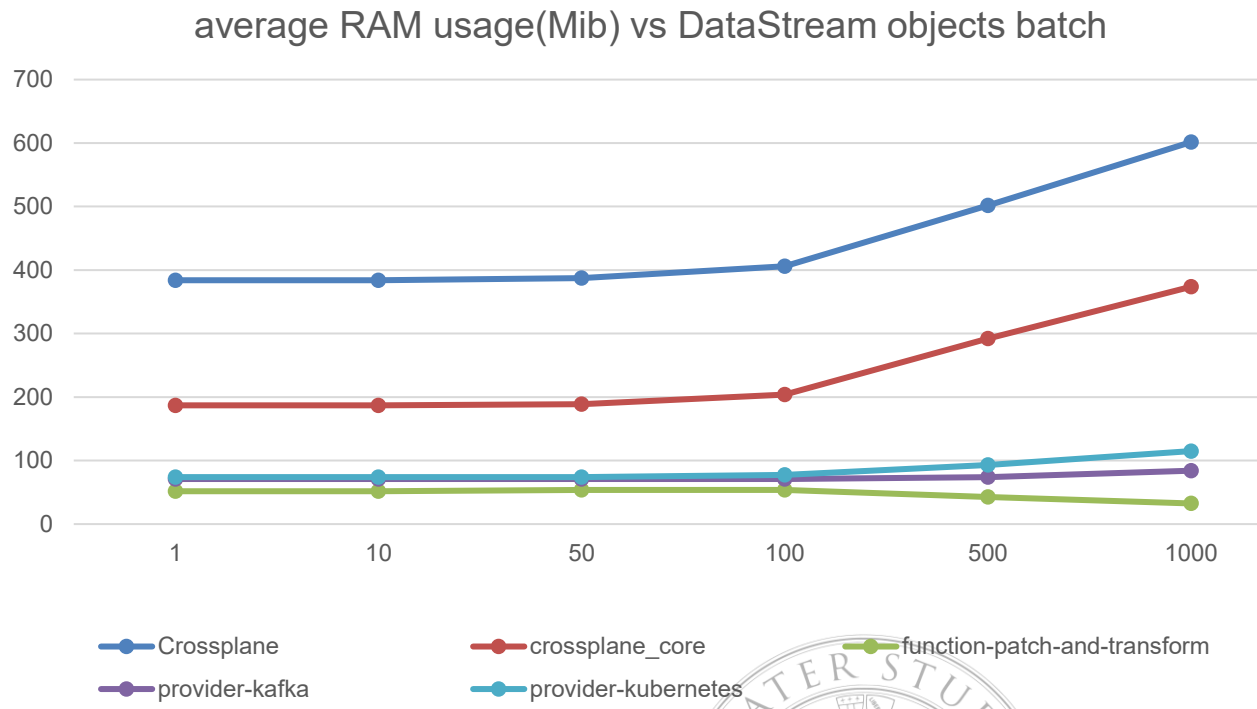




X-axis: batch size, the number of DataStream resources created together
Y-axis: average CPU rate (millicores)/RAM usage(Mib) each component sustained while it was actively reconciling that batch

*Crossplane measurements includes the full control plane (core + providers + functions).

# Resource consumption Crossplane




X-axis: batch size, the number of DataStream resources created together (N)
Y-axis: average CPU rate (millicores)/RAM usage(MiB) each component sustained while it was actively reconciling that batch

# REFERENCES


- The Kubebuilder Book, https://book.kubebuilder.io
- Crossplane documentation, https://docs.crossplane.io/v2.3
- kafka-go, https://github.com/segmentio/kafka-go


# Questions?

**Thank you for your attention**